How to create a fixed length array & immutable list :

---

1) Fixed length Array List :

java.util.Arrays class has provided a predefined static method called
asList(T ...values), It will create a fixed length array and the return type of this method is List interface.

public static <T> List<E> asList(T ...values)

In this fixed length array we can't perform add or remove opeartion otherwise we will get
java.lang.UnsupportedOperationException but we can replace the existing element by using set(int index, E obj)

//Program

```java
package com.ravi.vector_demo;

import java.util.Arrays;
import java.util.List;

public class FixedLengthArray
{
   public static void main(String[] args)
   {
      List<Integer> listOfNumbers = Arrays.asList(1,2,3,4,5,6,7,8);

      //listOfNumbers.add(9);      //java.lang.UnsupportedOperationException
      //listOfNumbers.remove(0); //java.lang.UnsupportedOperationException
      listOfNumbers.set(1, 100);
      listOfNumbers.forEach(num -> IO.print(num+" "));

   }
}
```

2) Immutable List : [Read only operation]

List interface has provided a predefined static method called
of(E ...x) available from java 9V.

public static <E> List<E> of(E... elements)

It will create an immutable list (Un-modifiable list), return type of this method is List<E>. Once it is created after that we can't pefrom any kind of operation like add(), remove() or replace [set(int index, Object obj)] otherwise we will get java.lang.UnsupportedOperationException

```java
package com.ravi.vector_demo;

import java.util.List;
record Stud(Integer id) {}

public class ImmutableList
{
    public static void main(String[] args)
    {
        List<Integer> listOfNumbers = List.of(1,2,3,4,5,6,7,8,9,10,11);

        listOfNumbers.add(12);//java.lang.UnsupportedOperationException
        listOfNumbers.remove(0);//java.lang.UnsupportedOperationException
        listOfNumbers.set(0, 100); //java.lang.UnsupportedOperationException

        listOfNumbers.forEach(IO::println);

    }
}
```

Stack<E>

public class Stack<E> extends Vector<E>

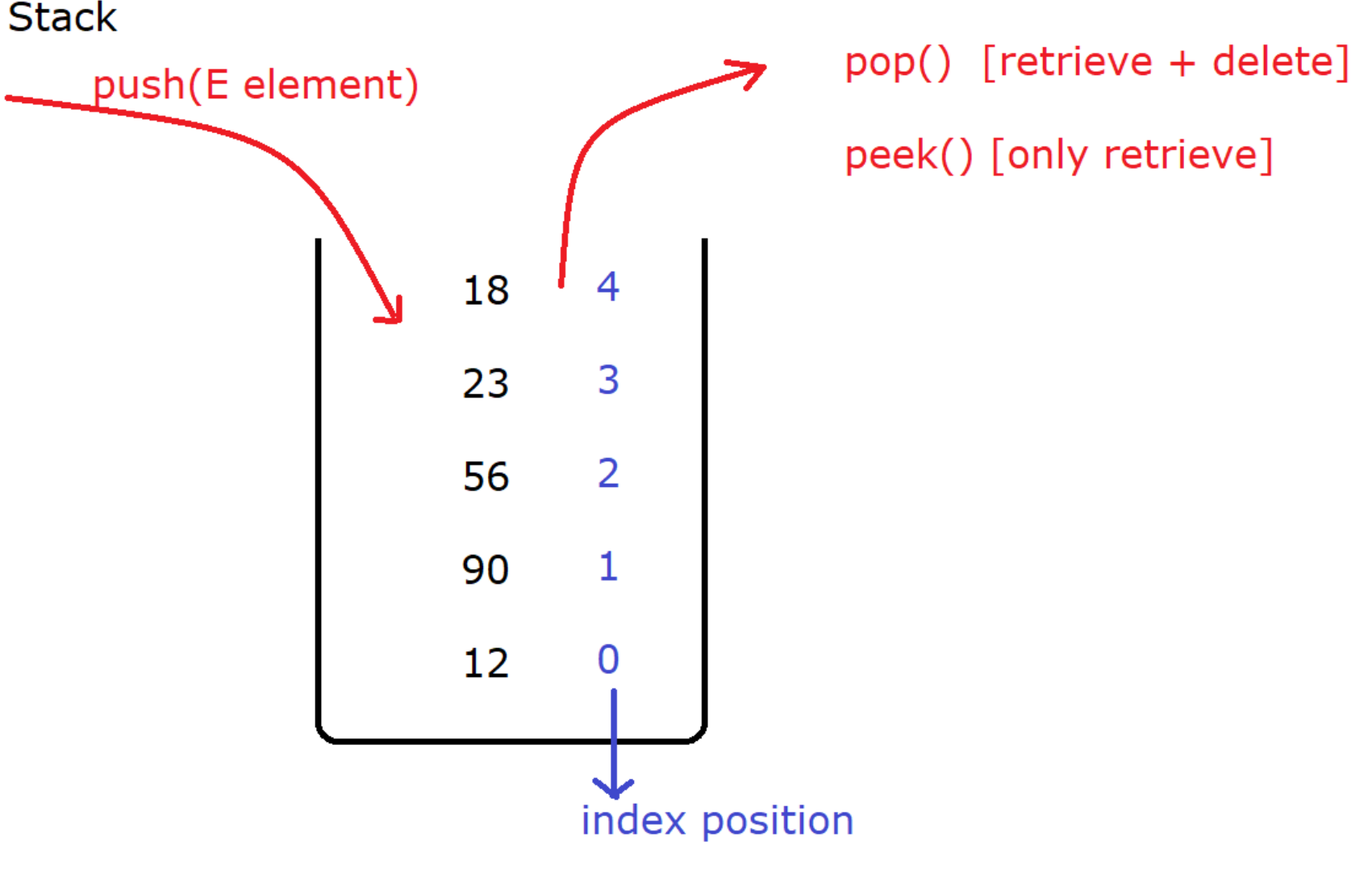
It is a predefined class available in java.util package. It is the sub class of Vector class introduced from JDK 1.0 so, It is also a legacy class.

It is a linear data structure that is used to store the Objects in LIFO (Last In first out) order.

Inserting an element into a Stack is known as push operation where as extracting an element from the top of the stack is known as pop operation.

Example :

Insert the following elements in the Stack
12, 90, 56, 23, 18



It throws an exception called java.util.EmptyStackException, if Stack is empty and we want to fetch the element.

It is a sub class of Vector, so It is dynamically Growable using Vector class constructor.

It has only one constructor as shown below

Stack s = new Stack();

Stack constructor internally invokes Vector constructor using super(), which creates a dynamic array of default capacity 10.

Methods :

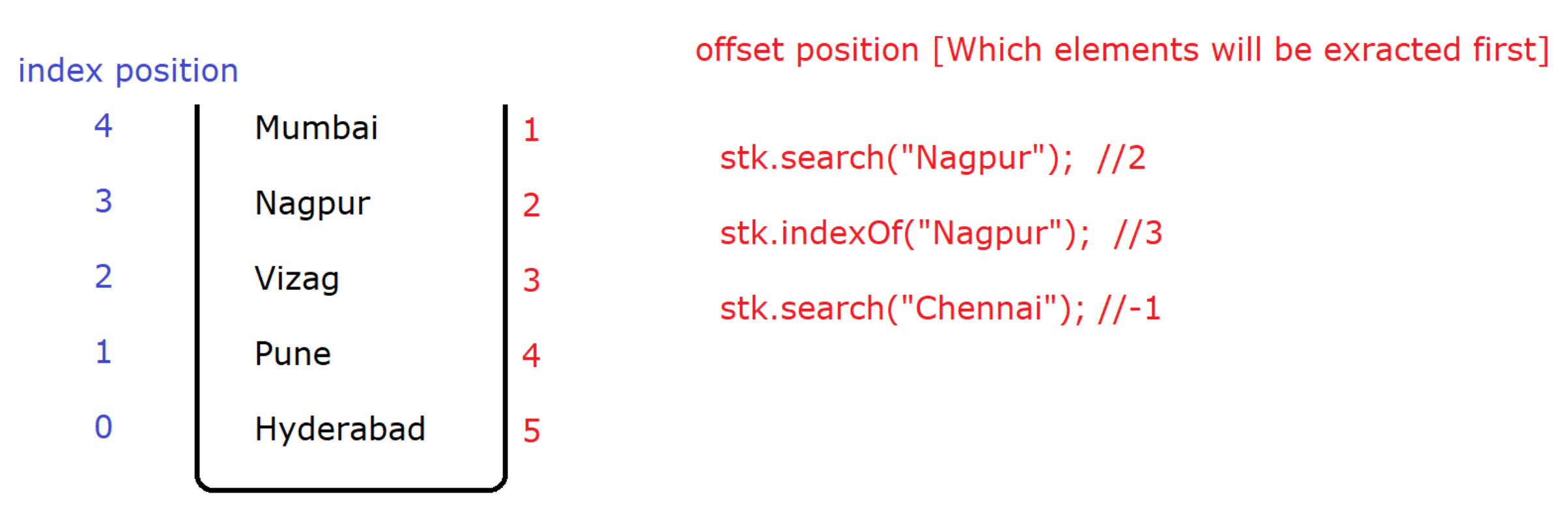
public E push(Object o) :- To insert an element in the top of the Stack internally using addElement() method.

public E pop() :- To remove and return the element from the top of the Stack.

public E peek() :- Will fetch the element from top of the Stack without removing.

public boolean empty() :- Verifies whether the stack is empty or not (return type is boolean)

public int search(Object o) :- It will search a particular element in the Stack and it returns OffSet position (int value). If the element is not present in the Stack it will return -1



stk.search("Nagpur"); //2

stk.indexOf("Nagpur"); //3

stk.search("Chennai"); //-1

//Programs :

```java
 package com.ravi.stack;

 import java.util.EmptyStackException;
 import java.util.Stack;

 public class StackDemo1
 {
     public static void main(String args[])
     {
         Stack<Integer> s = new Stack<>();

          try
          {
             s.push(12);
             s.push(15);
                   s.push(22);
                   s.push(33);
                   s.push(49);
                   System.out.println("After insertion elements are :"+s.toString());

             System.out.println("Fetching the elements using pop method");
             System.out.println(s.pop());
             System.out.println(s.pop());
             System.out.println(s.pop());
             System.out.println(s.pop());
             System.out.println(s.pop());



                   System.out.println("After deletion elements are :"+s);//[]

                   System.out.println("Is the Stack empty ? :"+s.empty());
          }
              catch(EmptyStackException e)
              {
                e.printStackTrace();
              }

     }
 }
```

```java
package com.ravi.stack;

import java.util.Stack;

//Reverse the String

public class StackDemo2
{
      public static void main(String args[])
      {
         Stack<Character> stack = new Stack<>();

         String str = "JavaTech";
         IO.println("Original String :"+str);

         char[] chars = str.toCharArray();

          for(char c :chars)
          {
              stack.push(c);
          }

          while(!stack.isEmpty())
          {
              IO.print(stack.pop()+" ");
          }


      }
}
```

```java
package com.ravi.stack;

import java.util.Stack;

public class StackDemo3
{
    public static void main(String[] args)
        {
            Stack<String> stk= new Stack<>();
            stk.push("Apple");
            stk.push("Grapes");
            stk.push("Mango");
            stk.push("Orange");
            IO.println("Stack: " + stk);

            String fruit = stk.peek();
            IO.println("Element at top: " + fruit);
            IO.println("Stack elements are : " + stk);
        }
}
```

```java
package com.ravi.stack;

import java.util.Stack;

public class StackDemo4
{
    public static void main(String[] args) //
        {

            Stack<String> stk= new Stack<>();
            stk.push("Apple");
            stk.push("Grapes");
            stk.push("Mango");
            System.out.println("Offset Position is : " + stk.search("Mango")); //1
            System.out.println("Offser Position is : " + stk.search("Banana")); //-1
            System.out.println("Is stack empty ? "+stk.empty()); //false
            System.out.println("Index Position is : " + stk.indexOf("Mango"));  //2

        }
}
```